Guia de configuração – Pterodactyl (Painel + Wings) para CS2/CSGO

Este documento descreve como configurar o projeto **cs2-csgo** (Pterodactyl Panel + Wings) em:

1. **Ambiente local** – outro PC com Windows e Docker Desktop
2. **AWS EC2** – uma ou mais instâncias Linux com Docker
3. **AWS ECS** – painel e Wings em tarefas Fargate/EC2
4. **AWS EKS** – cluster Kubernetes com deployments do painel e Wings

Para gerar um PDF: veja COMO-GERAR-PDF.md ou execute `.\generate-pdf.ps1` (se tiver pandoc).

Índice

- Pré-requisitos gerais
- 1. Setup local (outro PC)
- 2. AWS EC2
- 3. AWS ECS
- 4. AWS EKS
- Problemas comuns

Pré-requisitos gerais

- **Docker** e **Docker Compose** (v2+)
- **Git** (para clonar o repositório)
- Para **Windows local:** Docker Desktop com motor **WSL2** e, opcionalmente, **WSL2** com pasta do projeto acessível
- Para **AWS:** conta AWS, AWS CLI configurado (opcional), e conhecimentos básicos de VPC, segurança e domínio

---

1. Setup local (outro PC)

Objetivo: rodar o painel e o Wings no **Windows** (Docker Desktop) em outro computador, com acesso ao painel em `http://localhost` e Console do servidor a funcionar no browser.

1.1 Requisitos no PC

- **Windows 10/11** com Docker Desktop instalado
- Docker Desktop com **WSL2** como motor
- Portas **80, 443, 8080, 2022** livres (ou altere no compose)

1.2 Clonar o projeto e criar `.env`

```
git clone <url-do-repositorio> cs2-csgo
cd cs2-csgo
```

Copie o ficheiro de exemplo e preencha as variáveis:

```
copy .env.example .env
notepad .env
```

Preencha obrigatoriamente:

- `MYSQL_ROOT_PASSWORD` – senha root do MariaDB (ex.: `root123`)
- `MYSQL_PASSWORD` – senha do utilizador do painel (ex.: `panel123`)
- `APP_URL` – para local use `http://localhost`
- `APP_SERVICE_AUTHOR` – e-mail do admin (ex.: `admin@localhost`)

Para o Wings (depois de criar o node no painel), pode preencher:

- `WINGS_API_HOST=http://panel` (quando o Wings estiver na rede do painel)
- `WINGS_TOKEN_ID` e `WINGS_SYSTEM_TOKEN` – copiados do painel em **Admin** → **Nodes** → **[node]** → **Configuration**

1.3 Criar diretórios e patch do painel

- **Patch Disk I/O (Windows):** o projeto inclui `panel-patch/Server.php` montado no painel para permitir **Disk I/O = 0** e evitar o erro `io.weight` no Docker/WSL2. O `docker-compose-painel.yml` já monta este ficheiro; não remova a pasta `panel-patch` .

- **WSL2 – diretório para instalação de servidores:** no PowerShell execute uma vez:

```
wsl -e sh -c "mkdir -p /tmp/pterodactyl"
```

Isto cria `/tmp/pterodactyl` no host do Docker (WSL2), necessário para o Reinstall dos servidores.

1.4 Subir o painel

```
docker compose -f docker-compose-painel.yml up -d
```

Aguarde os containers (database, redis, panel). Na primeira vez a imagem do painel é construída a partir de `panel/` .

- Aceda a **http://localhost** e crie a primeira conta (admin).

1.5 Criar o node e obter tokens

1. **Admin** → **Nodes** → **Create Node**
   - Nome: por exemplo `local`
   - FQDN: `wings` (será usado com o Wings na mesma rede Docker)
   - Daemon Port: **8080**
   - **Use HTTP Connection** (não SSL para local)
2. Guarde e abra o node → aba **Configuration**
3. Copie **Token ID** e **Token** (ou o bloco de configuração) para o `.env` :
   - `WINGS_TOKEN_ID=...`
   - `WINGS_SYSTEM_TOKEN=...`
4. Opcional: em **Configuration** pode clicar em **Generate Token** e substituir o conteúdo de `wings-config/config.yml` pelo gerado; depois ajuste em `config.yml` apenas `api.host` para `0.0.0.0` e

`api.port: 8080` conforme a documentação do projeto.

1.6 Subir o Wings na rede do painel

Para o painel e o browser conseguirem falar com o Wings:

```
docker compose -f docker-compose-wings-windows.yml -f docker-compose-wings-windows-internal.yml up -d --force-recreate wings
```

Isto coloca o container **wings** na rede **pterodactyl_net** (junto do painel), acessível como **wings:8080**.

1.7 Ficheiro hosts para o Console no browser

O painel usa o FQDN do node (**wings**) para o WebSocket do Console. O browser precisa de resolver **wings** para o teu PC:

1. Abra **Bloco de notas como Administrador**
2. Ficheiro → Abrir → `C:\Windows\System32\drivers\etc` → tipo **Todos os ficheiros** → abra **hosts**
3. No final do ficheiro adicione:

   ```
   127.0.0.1 wings
   ```

4. Guarde e feche.

Assim, ao abrir o Console de um servidor, o browser liga a **ws://wings:8080/...** que resolve para **127.0.0.1:8080** (Wings no teu PC).

1.8 Resumo dos comandos locais

Ação	Comando
Subir painel	`docker compose -f docker-compose-painel.yml up -d`
Subir Wings (rede interna)	`docker compose -f docker-compose-wings-windows.yml -f docker-compose-wings-windows-internal.yml up -d`
Parar painel	`docker compose -f docker-compose-painel.yml down`
Ver logs do Wings	`docker compose -f docker-compose-wings-windows.yml logs wings -f`

Documentação adicional: `docs/erro-conectando-node.md`, `docs/wings-docker-windows.md`, `docs/rodar-local.md`.

---

2. AWS EC2

Objetivo: painel e Wings a correr em **uma ou duas instâncias EC2** com Docker (Linux).

2.1 Arquitetura sugerida

- **Opção A (uma instância):** 1 EC2 (Amazon Linux 2 ou Ubuntu) com Docker; painel + MariaDB + Redis + Wings no mesmo host; domínio ou IP público na porta 80/443.
- **Opção B (duas instâncias):** EC2-1 = painel + DB + Redis; EC2-2 = Wings (mais recursos para os jogos). O painel comunica com o Wings pelo IP privado ou nome interno (ex.: DNS privado).

2.2 Preparar a instância (Amazon Linux 2 ou Ubuntu)

```
# Instalar Docker (Amazon Linux 2)
sudo yum update -y
sudo yum install -y docker
sudo systemctl start docker
sudo systemctl enable docker
sudo usermod -aG docker ec2-user

# Instalar Docker Compose v2
sudo curl -L "https://github.com/docker/compose/releases/latest/download/docker-compose-$(uname -s)-$(uname -m)" -o /usr/local/bin/docker-compose
sudo chmod +x /usr/local/bin/docker-compose
```

Para Ubuntu use `apt` e o repositório oficial do Docker.

2.3 Clonar o projeto e configurar

```
git clone <url-do-repositorio> cs2-csgo
cd cs2-csgo
cp .env.example .env
nano .env
```

Defina:

- `MYSQL_ROOT_PASSWORD` e `MYSQL_PASSWORD`
- `APP_URL` – URL pública (ex.: `https://painel.seudominio.com` ou `http://<IP-EC2>`)
- `APP_SERVICE_AUTHOR` – e-mail do admin
- Se o Wings estiver na mesma máquina: no painel, node com FQDN = `localhost` ou IP privado e porta 8080.

2.4 Diretórios no host (Linux)

```
sudo mkdir -p /var/lib/pterodactyl/volumes /run/wings/machine-id /tmp/pterodactyl
sudo chown -R 988:988 /var/lib/pterodactyl /run/wings/machine-id /tmp/pterodactyl
```

2.5 Subir serviços

Só painel (primeira vez):

```
docker compose -f docker-compose-painel.yml up -d
```

Depois de criar o node no painel e obter o token, configure o Wings (em `/etc/pterodactyl/config.yml` ou use o `wings-config/` do projeto) e suba o Wings. No Linux pode usar `docker-compose-wings.yml` (sem o override Windows).

2.6 Segurança e rede AWS

- **Security Group da EC2:** permitir entrada em 80 (HTTP), 443 (HTTPS), 8080 (Wings API, se for acessível externamente), 2022 (SFTP), e as portas dos jogos (ex.: 27015 para CS2).
- **SSL:** use Let's Encrypt com Nginx (ex.: `docker-compose-painel.npm.yml` ou proxy reverso) ou um Certificate Manager + ALB em frente à EC2.
- Para produção, coloque a base de dados e o Redis em serviços geridos (RDS, ElastiCache) e ajuste o compose para apontar para eles em vez de containers locais.

---

3. AWS ECS

Objetivo: painel e Wings como **tarefas ECS** (Fargate ou EC2 launch type), com balanceador de carga e, opcionalmente, RDS/ElastiCache.

3.1 Conceitos

- **Cluster ECS** – agrupa os serviços.
- **Task Definition** – define imagem, CPU/memória, variáveis de ambiente, volumes e rede.
- **Service** – mantém N tarefas em execução (painel, Wings, etc.).
- **ALB (Application Load Balancer)** – termina SSL e encaminha tráfego para o painel (e, se quiser, para o Wings na porta 8080).

3.2 Passos gerais

1. **Criar o cluster ECS** (consola AWS ou Terraform/CloudFormation).
2. **Repositório ECR** – construir e enviar imagens do painel (e, se usar imagem própria do Wings, também):

   ```
   aws ecr get-login-password --region sa-east-1 | docker login --username AWS --
   password-stdin <account-id>.dkr.ecr.sa-east-1.amazonaws.com
   docker build -t pterodactyl-panel ./panel
   docker tag pterodactyl-panel:latest <account-id>.dkr.ecr.sa-east-
   1.amazonaws.com/pterodactyl-panel:latest
   docker push <account-id>.dkr.ecr.sa-east-1.amazonaws.com/pterodactyl-panel:latest
   ```

3. **Task Definition – Painel:**
   - Imagem: ECR do painel (ou `ghcr.io/pterodactyl/panel` se não customizar).
   - Variáveis: `APP_URL`, `DB_HOST` (RDS ou nome do serviço ECS do DB), `DB_PASSWORD`, `REDIS_HOST`, etc.
   - Porta 80 (ou 443 se terminar SSL na tarefa).
   - Logs: CloudWatch Logs.
4. **Task Definition – Database/Redis:** pode usar tarefas ECS com imagens `mariadb:10.5` e `redis:alpine`, ou preferir **RDS** e **ElastiCache**.
5. **Task Definition – Wings:**
   - Imagem: `ghcr.io/pterodactyl/wings:latest`.
   - Variáveis: configurar via env ou Secrets Manager (token do painel).
   - **Importante:** o Wings precisa de acesso ao **Docker daemon** para criar containers dos servidores de jogo. Em Fargate isso **não** é possível (não há Docker-in-Docker suportado). Por isso, o Wings deve correr em **ECS com launch type EC2** (instâncias EC2 no cluster com Docker instalado e agent ECS) ou numa **EC2 separada** com Docker.
6. **Service – Painel:** tipo Fargate ou EC2; desired count 1; ALB como target group na porta 80/443.
7. **Service – Wings:** apenas com launch type **EC2**; instâncias com Docker e socket exposto para o container Wings (configuração avançada de volumes e segurança).

3.3 Resumo ECS

- **Painel + DB + Redis:** podem ser Fargate (ou EC2) e expostos via ALB.
- **Wings:** deve correr em **EC2** (no cluster ECS com launch type EC2 ou numa EC2 dedicada), com Docker no host e segurança de rede (portas 8080, 2022, portas dos jogos) e integração com o painel via URL interna ou VPC.

---

4. AWS EKS

Objetivo: painel e Wings em **Kubernetes** (EKS), com Ingress e, opcionalmente, RDS/ElastiCache.

4.1 Conceitos

- **Deployments** – painel, Wings, MariaDB, Redis.
- **Services** – ClusterIP (interno) ou LoadBalancer/NodePort para expor.
- **Ingress** – terminação SSL e encaminhamento para o painel (e, se desejado, para o Wings na porta 8080).
- **PersistentVolumeClaim** – para dados do MariaDB e volumes do Wings (`/var/lib/pterodactyl/volumes`).

4.2 Recursos principais

- **Namespace** – por exemplo `pterodactyl` .
- **Secrets** – senhas do MySQL, token do Wings, etc.
- **ConfigMap** – trechos de configuração (ex.: `wings-config`).
- **Deployment – painel:** imagem do painel; env a apontar para o Service do MySQL e do Redis; volume para storage de logs e ficheiros.
- **Deployment – Wings:** imagem `ghcr.io/pterodactyl/wings` ; **privileged** e montagem de `/var/run/docker.sock` (ou o daemon Docker do node) para criar containers de jogos. Isto exige que o Wings rode em **nodes que tenham Docker** (ex.: nodes geridos com Docker instalado) ou solução tipo DinD (Docker-in-Docker) com cuidado de segurança.
- **Deployment – MariaDB e Redis:** imagens oficiais; PVC para dados.
- **Ingress:** host `painel.seudominio.com` → Service do painel; opcionalmente host/path para a API do Wings na porta 8080.

4.3 Exemplo mínimo (conceito)

```yaml
# Exemplo de Deployment do painel (resumido)
apiVersion: apps/v1
kind: Deployment
metadata:
  name: pterodactyl-panel
  namespace: pterodactyl
spec:
  replicas: 1
  selector:
    matchLabels:
      app: panel
  template:
    metadata:
      labels:
        app: panel
    spec:
```

```yaml
      containers:
      - name: panel
        image: <ecr ou ghcr.io>/pterodactyl-panel:latest
        ports:
        - containerPort: 80
        env:
        - name: APP_URL
          value: "https://painel.seudominio.com"
        - name: DB_HOST
          value: "mariadb"
        - name: DB_PASSWORD
          valueFrom:
            secretKeyRef:
              name: panel-secrets
              key: db-password
        # ... mais env e volumeMounts
```

O Wings em EKS precisa de **nodes com Docker** e de montar o socket do Docker; a definição exata depende da sua topologia (nodes dedicados a jogos, tolerations, etc.).

4.4 Resumo EKS

- **Painel, DB, Redis:** deployments + services + Ingress; pode usar Helm para empacotar.
- **Wings:** deployment em nodes que tenham Docker (ou DinD) e acesso à rede do painel; PVC para `/var/lib/pterodactyl/volumes` e `/tmp/pterodactyl`; expor 8080 e 2022 conforme necessário.

Problemas comuns

Problema	Solução
"Could not establish a connection to the machine"	Verificar se o Wings está Up; node com FQDN correto (local: `wings` + entrada no hosts; EC2: IP ou DNS interno).
Reinstall dá 500	Verificar se o painel alcança o Wings (curl de dentro do container do painel); ver `docs/erro-conectando-node.md`.
`io.weight` / container do servidor não arranca	Usar **Disk I/O = 0** no painel (patch em `panel-patch/Server.php`); em Linux puro pode usar 10–1000.
"bind source path does not exist" no Reinstall	No Windows/WSL2, criar `/tmp/pterodactyl` no host (WSL2) e montar no Wings; ver `docker-compose-wings-windows.yml`.
Permission denied em `laravel.log`	Usar volume nomeado `panel_logs` para `/app/storage/logs` (já no `docker-compose-painel.yml`).
Console no browser não liga	FQDN do node = `wings`; adicionar `127.0.0.1 wings` ao ficheiro **hosts** do Windows (local).

Para mais detalhes, consulte a pasta **docs/** na raiz do projeto.

Como gerar o PDF deste guia

- **Opção 1:** Abra `README.md` no VS Code ou no GitHub e use **Imprimir** → **Guardar como PDF** (ou extensão "Markdown PDF").
- **Opção 2:** Com **pandoc** instalado:

  ```
  pandoc docs/setup/README.md -o docs/setup/setup-guia.pdf --pdf-engine=xelatex -V mainfont="DejaVu Sans"
  ```

- **Opção 3:** Use um conversor online (ex.: Markdown to PDF) com o conteúdo de `docs/setup/README.md`.

O ficheiro **setup-guia.pdf** pode ser gerado na pasta `docs/setup/` e partilhado em conjunto com o repositório.